Assignment 3: Understanding and Attempting the ARC-AGI Challenge

Course: Artificial Intelligence (CSE643)

⚠️ How this assignment is assessed (please read first)

You will NOT submit any code, report, or write-up for this assignment.

Your learning will be assessed through an **in-class, closed-book examination**. The exam will contain questions about the *content* you study (Parts 1 and 2) and about *your own implementation and what you learned from it* (Part 3). Questions will be answerable only if you genuinely did the work: read the sources, built the attempt, ran it, and thought about the results.

Please keep your code and any notes for your own use, and be ready to explain them from memory. Do not expect to look up your code during the exam.

1. Background and motivation

Most benchmarks in AI measure *skill* at a known task. In his 2019 paper *On the Measure of Intelligence*, François Chollet argues that skill is a poor proxy for intelligence, because skill can be bought with prior knowledge and enormous amounts of training data. He proposes instead that intelligence is **skill-acquisition efficiency**: how well a system learns to handle *new* tasks given limited experience and limited priors. The **Abstraction and Reasoning Corpus (ARC-AGI)** is his proposed benchmark for this idea.

This assignment asks you to study that idea, follow how the benchmark has evolved through three versions, and then get your hands dirty by writing a small solver for one of them, using the classical AI techniques you have learned in this course (search, planning, DSLs, simple reinforcement learning, etc.).

You are **not** expected to build a competitive system. Even the strongest systems in the world find these tasks hard. The goal is understanding.

2. Part 1: Study the foundations

Read Chollet's original paper:

> Chollet, F. (2019). *On the Measure of Intelligence.* arXiv:1911.01547.

You are not expected to digest every page of the formal treatment in the middle sections, but you should be able to discuss the following:

1. **How Chollet defines intelligence**, and why he argues that measuring skill alone is insufficient (consider "skill vs. generalization" and the role of priors and experience).
2. **The notion of Core Knowledge priors** (objectness, goal-directedness, numbers and counting, basic geometry and topology) and why ARC is designed to rely *only* on these.
3. **The structure of an ARC task**: demonstration input/output pairs, test input(s), grids of up to 30×30 with 10 colours, and the requirement that the exact output grid be produced.
4. **Why ARC tasks are hard for pattern-matching approaches** and why every task is meant to be novel.
5. **Limitations and criticisms** of the benchmark, as you see them. (We will not hand you a list; form your own view.)

Suggested supporting material: the ARC Prize website (arcprize.org), including its guides and the history page, and the official ARC repositories.

3. Part 2: The three versions of ARC-AGI

Study each of the following versions. For every version you should be able to describe **what it is, what it was designed to fix or test, how it is scored, and how the state of the art has evolved.**

3.1 ARC-AGI-1

- The original format (2019 paper; used in competitions from 2020 to 2024): static input/output grid puzzles.
- Typical size and structure of the training, public evaluation, and hidden evaluation sets.
- Past competitions (Kaggle 2020, ARCathon, ARC Prize 2024) and the kinds of approaches that led them (e.g. hand-crafted DSLs with brute-force program search, then LLM-based approaches with test-time adaptation).

3.2 ARC-AGI-2

- Introduced in 2025 as a harder, still-static successor.
- What changed in the task design (e.g. greater compositionality and need for contextual rule application) and why. Look for the explicit design goals, such as resisting brute-force search and reducing the chance that a task can be solved without real reasoning.
- The role of **efficiency** (compute/cost per task) in how results are reported and judged.
- The ARC Prize 2025 and 2026 competitions, and the concept of the **refinement loop** that characterised many of the top 2025 solutions.

3.3 ARC-AGI-3

- A **fundamentally different format**: *interactive* game-like environments rather than static puzzles. The agent receives no instructions and no stated goal; it must explore, infer the rules and the objective, and then act efficiently.
- What new capabilities this tests (exploration, memory, planning, goal inference over multiple steps) that static tasks cannot.
- How agents are **scored**, including the role of action efficiency relative to human performance.
- The history of the Developer Preview (2025), the 2026 launch, and what kinds of agents performed best in early evaluations.

3.4 Public vs. semi-private vs. private datasets

A central theme of this assignment. You must be able to explain:

- What the **public (training / evaluation)**, **semi-private**, and **private** sets are, and who has access to each.
- *Why* the distinction exists (think: data contamination, overfitting to a known test set, the cost of testing commercial models via API versus running code in a sandbox).
- Why scores on public sets and scores on private sets can differ substantially, and how to read a leaderboard critically.
- Why the official competitions run on Kaggle with **restricted conditions** (including constraints on internet access and compute), and how those conditions affect which approaches can compete there versus on the open leaderboard.

3.5 Tracking the state of the art

For each of the three versions, assemble a **timeline** of the state of the art in your own notes: roughly when scores reached what level, on which kind of dataset (public / semi-private / private), and which

type of approach achieved it.

> **Important:** this field moves quickly. In the exam you will be expected to know the overall trajectory, approximate scores of the three challenges over time and across the types of attempts (LLM or not), and *relationship* between public and private results, and what the trends tell us, rather than memorised numbers.

4. Part 3: Build a rudimentary attempt at ONE challenge

Choose **exactly one** of ARC-AGI-1, ARC-AGI-2, or ARC-AGI-3 and implement a small, working attempt. You will test it on **a few instances** (roughly 3 to 10 is plenty; more is not required and not rewarded).

4.1 Constraints

- **You have not covered machine learning or neural networks, and you should not rely on them.** The expected toolbox is the one from this course.
- You may *additionally* use an LLM, for instance to propose candidate transformations or programs, but a purely "paste the puzzle into a chatbot" attempt is not sufficient. The core of your system should be something you designed and can explain.
- Code does not have to be elegant or general. It *does* have to be something you understand fully.
- Your attempt may fail on most or all instances. That is a perfectly acceptable outcome, provided you understand *why*.

4.2 What you should be able to explain (in the exam)

Regardless of which challenge you chose, you must be able to describe:

1. **Your problem formulation**: how you represented states, actions, goals (or how you handled the absence of a stated goal), and any heuristics or reward signals.
2. **Your algorithm** and why you picked it over alternatives from the course.
3. **Which instances you tested**, what happened on each, and *why* the results came out as they did.
4. **Failure analysis**: at least one concrete case where your approach failed, what exactly went wrong, and what would be required to fix it.

5. **Scaling and limits**: what would happen to your approach on harder tasks? (e.g. branching factor, search depth, state-space size, exploration cost.)
6. **Connection to Chollet's thesis**: does your system display *skill* on the tasks you gave it, or genuine *skill-acquisition efficiency*? What priors did you hard-code, and does that matter?
7. **If you used an LLM**: what it contributed, how you verified its output, and where it was unreliable.
8. **What you learned** about the benchmark and about the limits of classical AI techniques.

5. Academic integrity and use of tools

- Because assessment is in person, the real risk of copying is that you will be unable to explain what you did. Write and understand your own code.
- You may use LLMs and online resources to help you learn and to help you code. You remain fully responsible for understanding everything you built. The exam will probe this.
- If you adapt ideas from published solutions or open-source DSLs, be prepared to explain precisely what you took and how it works.

Remember: no submission is required. Your preparation is tested in the exam.